

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Debugging & Optimisation Task (Low)

Assessment Tool Weightage: 30%

Dr.G.Ayyappan

Schema Used in All Questions:

Student (RollNo, Name, DeptID, Age, Marks) | Department (DeptID, DeptName, HOD)

Sample Data (load this into MySQL before attempting the questions):

Department

DeptID	DeptName	HOD
1	Computer Science	Dr. Rao
2	Electronics	Dr. Iyer
3	Mechanical	Dr. Nair

Student

RollNo	Name	DeptID	Age	Marks
101	Aravind	1	20	85
102	Bhavya	1	21	78
103	Chandran	1	20	90
104	Deepak	2	22	60
105	Esha	2	21	55
106	Farhan	2	22	95
107	Gowri	3	20	40
108	Hari	3	21	45
109	Indra	3	20	68

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Identify and fix the error in the given SQL query that uses incorrect JOIN syntax and returns duplicate results. Given Query: <pre>SELECT s.Name, d.DeptName FROM Student s, Department d; -- Intent: list each student with their own department's name</pre>	BL3 – Apply	5
2	The following sub-query returns incorrect results due to a missing correlation. Debug and rewrite it correctly. Given Query: <pre>SELECT s.Name, s.DeptID, s.Marks FROM Student s WHERE s.Marks > (SELECT AVG(Marks) FROM Student); -- Intent: list students who score above their OWN department's average</pre>	BL3 – Apply	5
3	A given SQL view definition causes errors when queried. Identify the issue and rewrite a correct and optimized view. Given Query: <pre>CREATE VIEW DeptAvgMarks AS SELECT Name, DeptID, AVG(Marks) AS AvgMarks FROM Student GROUP BY DeptID; SELECT * FROM DeptAvgMarks;</pre>	BL3 – Apply	5
4	Debug the given relational algebra expression that produces incorrect output for a natural join operation. Given Query: <pre>$\pi(\text{Name, Age})(\text{Student}) \bowtie \text{Department}$ -- Intent: natural join of Student and Department on DeptID</pre>	BL3 – Apply	5
5	The SQL query below violates referential integrity. Identify the violation and modify the statements to enforce proper constraints. Given Query: <pre>CREATE TABLE Student (RollNo INT PRIMARY KEY, Name VARCHAR(50), DeptID INT, Age INT, Marks INT); -- Department table only has DeptID values 1, 2, 3 INSERT INTO Student (RollNo, Name, DeptID, Age, Marks) VALUES (110, 'Arun', 9, 20, 78);</pre>	BL3 – Apply	5

6	Given an inefficient SQL query with nested loops, rewrite it using JOIN to optimize performance. Given Query: <pre>SELECT s.Name, (SELECT d.DeptName FROM Department d WHERE d.DeptID = s.DeptID) AS DeptName FROM Student s;</pre>	BL6 – Create	5
7	The GROUP BY query below produces wrong aggregation results. Debug the query and explain the error. Given Query: <pre>SELECT DeptID, AVG(Marks) AS AvgMarks FROM Student GROUP BY RollNo;</pre>	BL3 – Apply	5
8	Identify and fix the logical error in a given SQL UPDATE statement that unintentionally modifies all rows in the table. Given Query: <pre>-- Intent: give 5 bonus marks to students in Department 2 only UPDATE Student SET Marks = Marks + 5;</pre>	BL3 – Apply	5
9	Optimize the given multi-table SQL query by restructuring sub-queries into JOINS and using appropriate indexes. Given Query: <pre>SELECT s.Name, s.Marks, (SELECT DeptName FROM Department WHERE DeptID = s.DeptID) AS DeptName, (SELECT HOD FROM Department WHERE DeptID = s.DeptID) AS HeadOfDept FROM Student s WHERE s.DeptID IN (SELECT DeptID FROM Department WHERE DeptName IN ('Computer Science', 'Electronics'));</pre>	BL6 – Create	5
10	Debug a given relational calculus expression that returns an empty result set due to incorrect quantifier usage. Given Query: <pre>-- Intent: students who scored more than AT LEAST ONE student in Department 2 { s.Name Student(s) ∧ (∀ t) (Student(t) ∧ t.DeptID = 2 → s.Marks > t.Marks) }</pre>	BL3 – Apply	5

Code of Conduct:

*I **C.Sravanthi / 192525463** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: C.Sravanthi

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

ANSWERS:

Q1. Fix the Incorrect JOIN

Error:

- Cross join is used (FROM Student s, Department d), producing duplicate rows.

Correct SQL:

```
SELECT s.Name, d.DeptName
FROM Student s
JOIN Department d
ON s.DeptID = d.DeptID;
```

Q2. Fix the Missing Correlation

Error:

- It compares each student's marks with the overall average instead of their department average.

Correct SQL:

```
SELECT s.Name, s.DeptID, s.Marks
FROM Student s
WHERE s.Marks >
(
SELECT AVG(s2.Marks)
FROM Student s2
WHERE s2.DeptID = s.DeptID
);
```

Q3. Correct the View

Error:

- Name is not included in the GROUP BY clause.

Correct SQL:

```
CREATE VIEW DeptAvgMarks AS
SELECT DeptID,
AVG(Marks) AS AvgMarks
FROM Student
GROUP BY DeptID;
SELECT * FROM DeptAvgMarks;
```

Q4. Correct Relational Algebra

Error:

Natural join is performed after projecting only Name and Age, so DeptID is lost.

Correct Expression:

$\pi(\text{Name, Age, DeptName})$
 $(\text{Student} \bowtie \text{Department})$

or

$\pi(\text{Name, Age, DeptName})$
 $(\text{Student} \bowtie_{\text{Student.DeptID=Department.DeptID}} \text{Department})$

Q5. Referential Integrity

Error:

- DeptID = 9 does not exist in the Department table.

Correct SQL:

```
CREATE TABLE Student
(
  RollNo INT PRIMARY KEY,
  Name VARCHAR(50),
  DeptID INT,
  Age INT,
  Marks INT,
  FOREIGN KEY (DeptID)
  REFERENCES Department(DeptID)
);
```

Insert valid data:

```
INSERT INTO Student
VALUES
(110,'Arun',2,20,78);
```

Q6. Optimize Using JOIN

Original Query uses subquery.

Optimized Query:

```
SELECT s.Name,  
d.DeptName  
FROM Student s  
JOIN Department d  
ON s.DeptID = d.DeptID;
```

Q7. GROUP BY Error

Error:

- Grouping by RollNo calculates average for each student instead of each department.

Correct SQL:

```
SELECT DeptID,  
AVG(Marks) AS AvgMarks  
FROM Student  
GROUP BY DeptID;
```

Q8. UPDATE Statement

Error:

- All students receive bonus marks.

Correct SQL:

```
UPDATE Student  
SET Marks = Marks + 5  
WHERE DeptID = 2;
```

Q9. Optimize Multi-table Query

Optimized SQL:

```
SELECT  
s.Name,  
s.Marks,  
d.DeptName,  
d.HOD AS HeadOfDept  
FROM Student s  
JOIN Department d  
ON s.DeptID = d.DeptID  
WHERE d.DeptName IN  
('Computer Science','Electronics');
```

Recommended Index:

```
CREATE INDEX idx_student_dept  
ON Student(DeptID);  
CREATE INDEX idx_department_deptid  
ON Department(DeptID);
```

Q10. Relational Calculus**Error:**

- Universal quantifier ($\forall$) returns students scoring higher than **every** student in Department 2.

Correct Expression:

```
{ s.Name |  
Student(s)  $\wedge$   
( $\exists$  t)  
(Student(t)  $\wedge$   
t.DeptID = 2  $\wedge$   
s.Marks > t.Marks)  
}
```